

CHAPTER

4

Supplementary Principles

It is easier to fight for one's principles than to live up to them.

—Alfred Adler

Agile Modeling's supplementary principles define concepts that increase your effectiveness as an agile modeler. Although these principles support AM's core principles, described in Chapter 3, "Core Principles," their adoption is not required for you to be truly agile modeling. These principles are all very good ideas and you should adopt them if they fit well into your organizational culture. AM's supplementary principles are:

- Content is more important than representation
- Everyone can learn from everyone else
- Know your models
- Local adaptation
- Open and honest communication
- Work with people's instincts

Content Is More Important Than Representation

Any given model can be represented in several ways. For example: AUI specification could be created using Post-It notes on a large sheet of paper (an essential or low-

CHAPTER

4

Supplementary Principles

It is easier to fight for one's principles than to live up to them.

—Alfred Adler

Agile Modeling's supplementary principles define concepts that increase your effectiveness as an agile modeler. Although these principles support AM's core principles, described in Chapter 3, "Core Principles," their adoption is not required for you to be truly agile modeling. These principles are all very good ideas and you should adopt them if they fit well into your organizational culture. AM's supplementary principles are:

- Content is more important than representation
- Everyone can learn from everyone else
- Know your models
- Local adaptation
- Open and honest communication
- Work with people's instincts

Content Is More Important Than Representation

Any given model can be represented in several ways. For example: AUI specification could be created using Post-It notes on a large sheet of paper (an essential or low-

fidelity prototype); as a sketch on paper or a whiteboard; as a “traditional” prototype built using a prototyping tool or programming language; or as a formal document including both a visual representation as well as a textual description of the UI. Depending on the reason you’re creating the model, the various representations may be equivalent. If your goal is to explore the layout of an HTML page, then all three representations are sufficient. In fact, making the specification prettier using a drawing tool and documenting it comprehensively does not add anything to further this goal. The content is more important than its representation in this case.

An important implication of this principle is that you don’t need to jump into using a complicated CASE tool right away. Yes, CASE tools can be very useful if they generate code for you or if they can reverse engineer an understandable model from existing code, but at first it is better to work with simple, flexible tools. I discuss this in detail in Chapter 10, “Using the Simplest Tools Possible?”

Consider another example. A UML class diagram could be drawn as a sketch as shown in Figure 4.1, using a drawing tool such as Microsoft Visio as shown in Figure 4.2, or created using a sophisticated CASE tool. It’s still the same class diagram, just depicted differently. The sketch may be sufficient for its purpose—it helped the people that drew it to understand an initial approach to designing the software they were working on. Could you use the sketch of Figure 4.1 in your official documentation (assuming that it captures critical information that you want to persist)? Why not? It’s

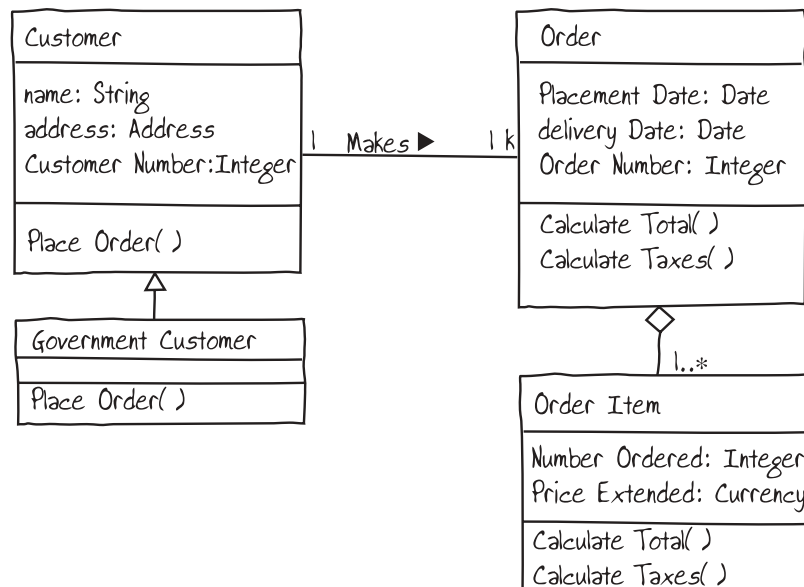


Figure 4.1 A UML class diagram sketch.

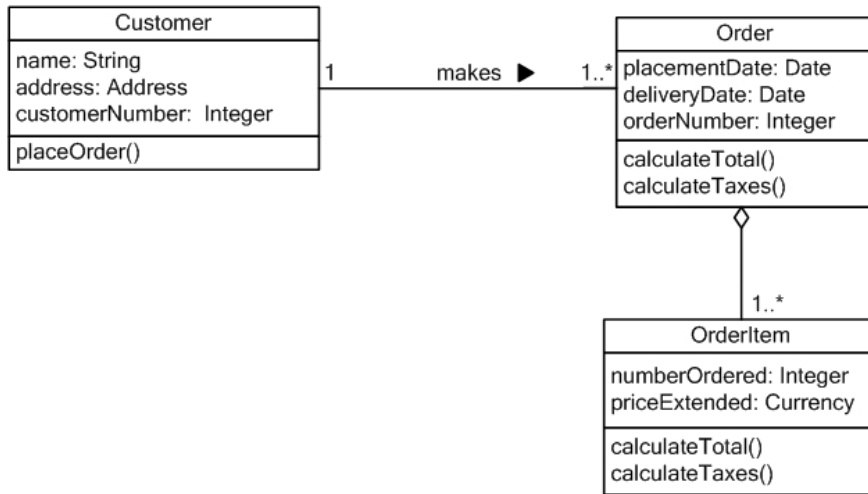


Figure 4.2 A UML class diagram using Microsoft Visio.

not as pretty as the tool-drawn diagram in Figure 4.2, but the sketch took one-third the time to draw. Remember, agile modelers *Maximize Stakeholder Investment*, so it's a serious decision to invest time simply to make something look nice. I suspect that the world isn't going to end if you use a hand-drawn sketch as part of your official documentation. I often use scans or digital pictures of sketches in official documents or presentations—the sketch captures the idea that I want to communicate and I don't want to waste time transcribing the sketch into a more sophisticated tool simply to make it look pretty. It's OK to show people imperfect work and to defend your decision to do only the minimal work required and then move on. This model clearly isn't complete, and frankly, my handwriting is a disaster, but remember, an agile modeler models with a purpose and stops as soon as that purpose is fulfilled.

TIP Meet the Expectations of Your Audience

Whether it is a viable strategy to include hand-drawn diagrams in your documentation depends on the audience: if your readers are “prim and proper,” then you'll likely be required to invest the time to transcribe your models using an electronic tool. There's nothing wrong with this; after all it's their money you're spending, but recognize that you do have an option to be more agile and that you should make them aware of this option.

Similarly, the same structure of classes presented in Figures 4.1 and 4.2 could be depicted using a different type of artifact, such as a Class Responsibility Collaborator (CRC) model or using a different notation, perhaps the OMT notation (Rumbaugh et al. 1991). Having said that, I would not choose the OMT notation over the UML notation for the simple reason that the UML is an industry standard, whereas the OMT notation is not. Yes, there may be technical reasons why OMT is superior to UML, but

chances are good that the communication loss from not using a common standard is more significant.

An interesting implication is that a model does not always need to be a document. I could have drawn the sketch in Figure 4.1 simply to understand the structure of the classes that I was working on, proceeded to code and test the classes, and then discarded the model once I was finished with it. Even a complex set of diagrams created using a CASE tool may not become part of a document; instead they are used as inputs into other artifacts, very likely source code, but never formalized as official documentation. The point is that you take advantage of the benefits of modeling without incurring the costs of creating and maintaining documentation. This principle is motivated by AM's values of *Simplicity* and *Communication* and the Agile Alliance's (2001a) preference for working software over comprehensive documentation.

Everyone Can Learn from Everyone Else

Agile modelers recognize that they can never truly master something; there is always an opportunity to learn more and to extend your knowledge. They take the opportunity to work with and learn from others, to try new ways of doing things, to reflect on what seems to work and what doesn't. Technologies change rapidly; existing technologies such as Java evolve at a blinding pace, and new technologies such as C# and .NET are introduced regularly. Existing development techniques evolve at a slower pace, but they still evolve. As an industry we've understood the fundamentals of testing for quite some time, although we are constantly improving our understanding through research and practice. The point is that we work in an industry where change is the norm, where you must take every opportunity to learn new ways of doing things through training, education, mentoring, reading, and *working with each other*.

An implication of this practice is that everyone should expect to work with others to help them learn new skills. In fact, I believe that you are responsible to help others increase their intellectual toolkit. This principle is motivated by the values of *Communication* and *Humility*.

Know Your Models

Because you have multiple models that you can apply as an agile modeler, you need to know their strengths and weaknesses to be effective in their use. Furthermore, modeling techniques evolve all the time to reflect the changes in technology. As aspect-oriented programming (Xerox 2001) becomes more mainstream, if it ever does, I fully expect to have to learn one or more aspect-oriented models and/or aspect-oriented extensions to existing types of models. This enables you to keep your models as simple as possible as well as improve the quality of communication on the project that you're applying the modeling techniques to.

chances are good that the communication loss from not using a common standard is more significant.

An interesting implication is that a model does not always need to be a document. I could have drawn the sketch in Figure 4.1 simply to understand the structure of the classes that I was working on, proceeded to code and test the classes, and then discarded the model once I was finished with it. Even a complex set of diagrams created using a CASE tool may not become part of a document; instead they are used as inputs into other artifacts, very likely source code, but never formalized as official documentation. The point is that you take advantage of the benefits of modeling without incurring the costs of creating and maintaining documentation. This principle is motivated by AM's values of *Simplicity* and *Communication* and the Agile Alliance's (2001a) preference for working software over comprehensive documentation.

Everyone Can Learn from Everyone Else

Agile modelers recognize that they can never truly master something; there is always an opportunity to learn more and to extend your knowledge. They take the opportunity to work with and learn from others, to try new ways of doing things, to reflect on what seems to work and what doesn't. Technologies change rapidly; existing technologies such as Java evolve at a blinding pace, and new technologies such as C# and .NET are introduced regularly. Existing development techniques evolve at a slower pace, but they still evolve. As an industry we've understood the fundamentals of testing for quite some time, although we are constantly improving our understanding through research and practice. The point is that we work in an industry where change is the norm, where you must take every opportunity to learn new ways of doing things through training, education, mentoring, reading, and *working with each other*.

An implication of this practice is that everyone should expect to work with others to help them learn new skills. In fact, I believe that you are responsible to help others increase their intellectual toolkit. This principle is motivated by the values of *Communication* and *Humility*.

Know Your Models

Because you have multiple models that you can apply as an agile modeler, you need to know their strengths and weaknesses to be effective in their use. Furthermore, modeling techniques evolve all the time to reflect the changes in technology. As aspect-oriented programming (Xerox 2001) becomes more mainstream, if it ever does, I fully expect to have to learn one or more aspect-oriented models and/or aspect-oriented extensions to existing types of models. This enables you to keep your models as simple as possible as well as improve the quality of communication on the project that you're applying the modeling techniques to.

Local Adaptation

It is doubtful that you will be able to “apply AM out of the box.” Instead, you will need to modify it to reflect your environment, including the nature of your organization, your co-workers, your project stakeholders, and your project itself. When you adapt AM to meet your unique needs, you will definitely need to consider the modeling techniques that you plan to apply. For example: Your users may insist on concrete user interfaces instead of initial sketches or essential UI prototypes. The tools that you use will have an effect on your approach. Perhaps there isn’t a budget for a digital camera but you already have licenses for an existing CASE tool. AM can be modified to reflect the software process where it is applied. Part Three discusses applying AM on an XP project, and Part Four describes applying AM on a Unified Process project.

You will adapt your approach at the individual level as well as at the project level. For example: Some developers prefer one set of tools over another. For instance, when I’m Java coding, I prefer a sophisticated code editor and the JDK, whereas my co-workers may prefer a Java IDE. Some people focus on coding and do very little modeling, whereas others are more visually oriented and prefer to invest time sketching before they write code—different people, different applications of AM. It’s important to note that the values, principles, and practices remain the same, but their application varies.

Also, remember that AM isn’t going to work in all situations, as I indicated in Chapter 1, “Introduction.” It may not be realistic to try to adopt AM in full, at least not right away, so you may find that you can tailor a portion of AM’s principles and practices into your existing software process. As long as you improve your effectiveness as a developer, that’s perfectly fine.

Open and Honest Communication

People need to be free, and know that they are free, to offer suggestions. This includes ideas that pertain to one or more models. Perhaps someone has a new way to approach a portion of the design or has new insight regarding a requirement, or simply a new method to present the current status of their work. Open and honest communication enables people to make better decisions because those decisions are based on more accurate information.

Open and honest communication requires commitment on everyone’s part and an understanding that effective communication is a critical success factor for software development projects. People who dare to speak their mind must also be open to hearing something they may not like. It requires humility on everyone’s part to be willing to abandon their pet ideas upon hearing that they may not be as good as they originally thought.

Work with People’s Instincts

When someone feels that something isn’t going to work, that a few things are inconsistent with one another, or that something doesn’t “smell right,” then there is a good

Local Adaptation

It is doubtful that you will be able to “apply AM out of the box.” Instead, you will need to modify it to reflect your environment, including the nature of your organization, your co-workers, your project stakeholders, and your project itself. When you adapt AM to meet your unique needs, you will definitely need to consider the modeling techniques that you plan to apply. For example: Your users may insist on concrete user interfaces instead of initial sketches or essential UI prototypes. The tools that you use will have an effect on your approach. Perhaps there isn’t a budget for a digital camera but you already have licenses for an existing CASE tool. AM can be modified to reflect the software process where it is applied. Part Three discusses applying AM on an XP project, and Part Four describes applying AM on a Unified Process project.

You will adapt your approach at the individual level as well as at the project level. For example: Some developers prefer one set of tools over another. For instance, when I’m Java coding, I prefer a sophisticated code editor and the JDK, whereas my co-workers may prefer a Java IDE. Some people focus on coding and do very little modeling, whereas others are more visually oriented and prefer to invest time sketching before they write code—different people, different applications of AM. It’s important to note that the values, principles, and practices remain the same, but their application varies.

Also, remember that AM isn’t going to work in all situations, as I indicated in Chapter 1, “Introduction.” It may not be realistic to try to adopt AM in full, at least not right away, so you may find that you can tailor a portion of AM’s principles and practices into your existing software process. As long as you improve your effectiveness as a developer, that’s perfectly fine.

Open and Honest Communication

People need to be free, and know that they are free, to offer suggestions. This includes ideas that pertain to one or more models. Perhaps someone has a new way to approach a portion of the design or has new insight regarding a requirement, or simply a new method to present the current status of their work. Open and honest communication enables people to make better decisions because those decisions are based on more accurate information.

Open and honest communication requires commitment on everyone’s part and an understanding that effective communication is a critical success factor for software development projects. People who dare to speak their mind must also be open to hearing something they may not like. It requires humility on everyone’s part to be willing to abandon their pet ideas upon hearing that they may not be as good as they originally thought.

Work with People’s Instincts

When someone feels that something isn’t going to work, that a few things are inconsistent with one another, or that something doesn’t “smell right,” then there is a good

Local Adaptation

It is doubtful that you will be able to “apply AM out of the box.” Instead, you will need to modify it to reflect your environment, including the nature of your organization, your co-workers, your project stakeholders, and your project itself. When you adapt AM to meet your unique needs, you will definitely need to consider the modeling techniques that you plan to apply. For example: Your users may insist on concrete user interfaces instead of initial sketches or essential UI prototypes. The tools that you use will have an effect on your approach. Perhaps there isn’t a budget for a digital camera but you already have licenses for an existing CASE tool. AM can be modified to reflect the software process where it is applied. Part Three discusses applying AM on an XP project, and Part Four describes applying AM on a Unified Process project.

You will adapt your approach at the individual level as well as at the project level. For example: Some developers prefer one set of tools over another. For instance, when I’m Java coding, I prefer a sophisticated code editor and the JDK, whereas my co-workers may prefer a Java IDE. Some people focus on coding and do very little modeling, whereas others are more visually oriented and prefer to invest time sketching before they write code—different people, different applications of AM. It’s important to note that the values, principles, and practices remain the same, but their application varies.

Also, remember that AM isn’t going to work in all situations, as I indicated in Chapter 1, “Introduction.” It may not be realistic to try to adopt AM in full, at least not right away, so you may find that you can tailor a portion of AM’s principles and practices into your existing software process. As long as you improve your effectiveness as a developer, that’s perfectly fine.

Open and Honest Communication

People need to be free, and know that they are free, to offer suggestions. This includes ideas that pertain to one or more models. Perhaps someone has a new way to approach a portion of the design or has new insight regarding a requirement, or simply a new method to present the current status of their work. Open and honest communication enables people to make better decisions because those decisions are based on more accurate information.

Open and honest communication requires commitment on everyone’s part and an understanding that effective communication is a critical success factor for software development projects. People who dare to speak their mind must also be open to hearing something they may not like. It requires humility on everyone’s part to be willing to abandon their pet ideas upon hearing that they may not be as good as they originally thought.

Work with People’s Instincts

When someone feels that something isn’t going to work, that a few things are inconsistent with one another, or that something doesn’t “smell right,” then there is a good

chance that that is actually the case. As you gain experience at developing software, your instincts become sharper, and what your instincts are telling you subconsciously can often be an important input into your modeling efforts. If your instincts tell you that a requirement doesn't make sense or it isn't complete, investigate it with your users. If your instincts tell you that a portion of your architecture isn't going to meet your needs, build a quick technical end-to-end prototype to test out your theory. If your instincts tell you that design alternative A is better than design alternative B, and there is no compelling reason to choose either one of them, then go with alternative A for now. It's important to understand that AM's *Courage* value indicates that you can remedy the situation at some point in the future if you discover your instincts were wrong.

Benefiting from These Principles

The principles described in this chapter and Chapter 3 are little better than well-intentioned exhortations unless you internalize them. Agile modelers adopt and act in accordance with these principles, something that is easy to say and hard to do. I don't know how to ensure that you actually internalize these principles; all I can do is ask that you think about them and try to follow them whenever you can. I've put together a pamphlet summarizing Agile Modeling—it includes a list of AM's values, principles, and practices—that you can download from the AM web site at www.agilemodeling.com. Perhaps printing this document, the three lists appearing on a single page, and tacking it on the wall in your work area will help your efforts. It's worth a try at least.

CHAPTER

5

Core Practices

Perfect practice makes perfect.

—Sensei Rick Willemsen

The heart of Agile Modeling is its practices. It is AM's practices that you will actually apply on your projects, practices that are guided by AM's values and principles. As with AM's principles, its practices are also organized into ones that are core and those that are supplementary. As I indicated in Chapter 3, "Core Principles," you must adopt all of AM's core practices to be able to claim that you are "doing AM." Of course, you can benefit from adopting only some of the practices, but it is better to adopt all of them if they fit well into your organization's culture. Adopting AM into your organization is discussed in detail in Chapter 28, "Adopting AM on an UP Project."

AM's core practices are organized into four categories that are described in detail in Chapter 7, "Order from Chaos: How The AM Practices Fit Together." These categories, and the core practices within them, are:

1. Iterative and Incremental Modeling
 - Apply the Right Artifact(s)
 - Create Several Models in Parallel
 - Iterate to Another Artifact
 - Model in Small Increments
2. Teamwork
 - Model with Others
 - Active Stakeholder Participation

- Collective Ownership
- Display Models Publicly

3. Simplicity

- Create Simple Content
- Depict Models Simply
- Use the Simplest Tools

4. Validation

- Consider Testability
- Prove it With Code

TIP None of These Practices Are New

As you read this chapter and Chapter 6, “Supplementary Practices,” you will very likely recognize many if not all of the practices that make up AM. That’s because the individual practices of AM aren’t new, they are techniques that effective modelers have been following for years. What is new is that I have packaged them together for the first time and described them in a single place. Furthermore, they are the distillation of hundreds of “best practices” that modelers follow, and as you’ll see in Chapter 7, they fit together into a synergistic whole, something that I call Agile Modeling.

Practices for Iterative and Incremental Modeling

AM defines four practices that support an iterative and incremental approach to modeling:

1. Apply the Right Artifact(s)
2. Create Several Models in Parallel
3. Iterate to Another Artifact
4. Model in Small Increments

Apply the Right Artifact(s)

This practice is AM’s equivalent of the adage “use the right tool for the job.” In this case, you want to create the right model(s) to get the job done. Each artifact—such as a UML state chart, an essential use case, source code, or data flow diagram (DFD)—has its own specific strengths and weaknesses, and therefore is appropriate for some situations but not others. For example: The UML activity diagram (Rumbaugh, Jacobson,

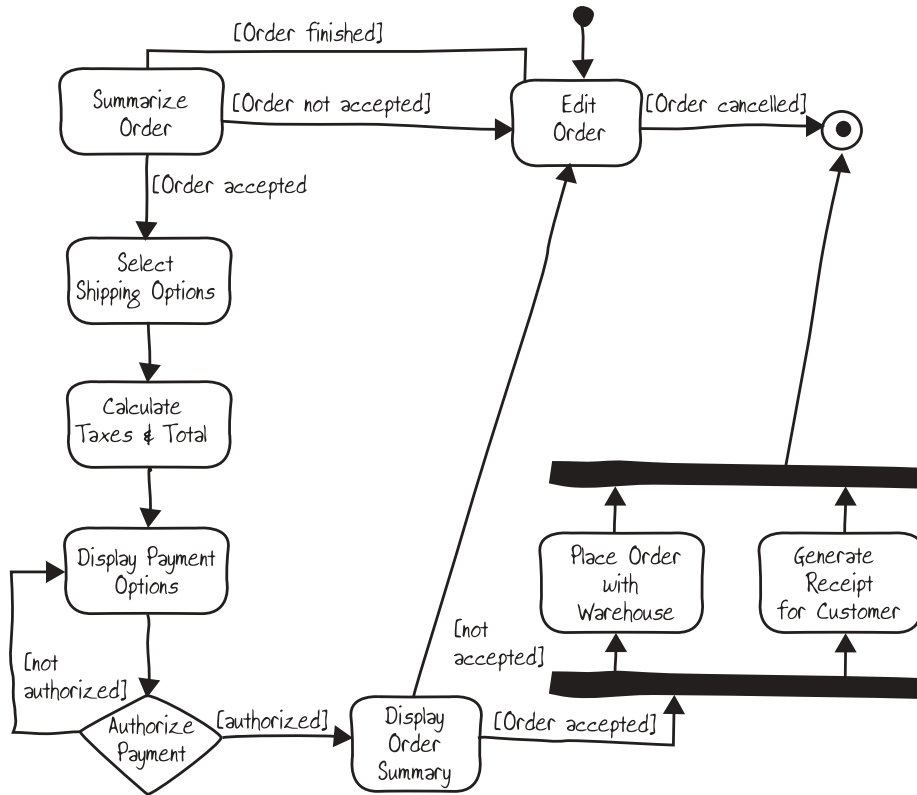


Figure 5.1 A UML Activity diagram for order processing.

and Booch 1999) in Figure 5.1 is useful for describing a business process, whereas the static structure of your database is better represented by a physical data model (Reingruber and Gregory 1994) such as the one presented in Figure 5.2. Similarly, a diagram is a better choice than source code. If a picture is worth a thousand words, then a model is often worth 1024 lines of code when applied in the right circumstances, an idea borrowed from Karl Wieggers (1999). This is because you can often explore design alternatives more effectively by drawing a couple of diagrams on whiteboards with your peers than you can by sitting down and developing code samples.

An important implication to agile modelers is that they need to gain an understanding of when and when not to apply each type of artifact, information that is presented in Appendix A, “Modeling Techniques,” for a wide range of modeling techniques. Learning the nuances of each type of artifact can be difficult, a problem that is compounded by the numerous artifacts available to you.

TIP Start By Learning a Subset of Artifacts

Overwhelmed by the number of modeling artifacts described in Appendix A? Don’t worry, that’s normal. A serious downside about AM is that it explicitly asks you to learn how to apply a wide range of modeling artifacts, and that’s

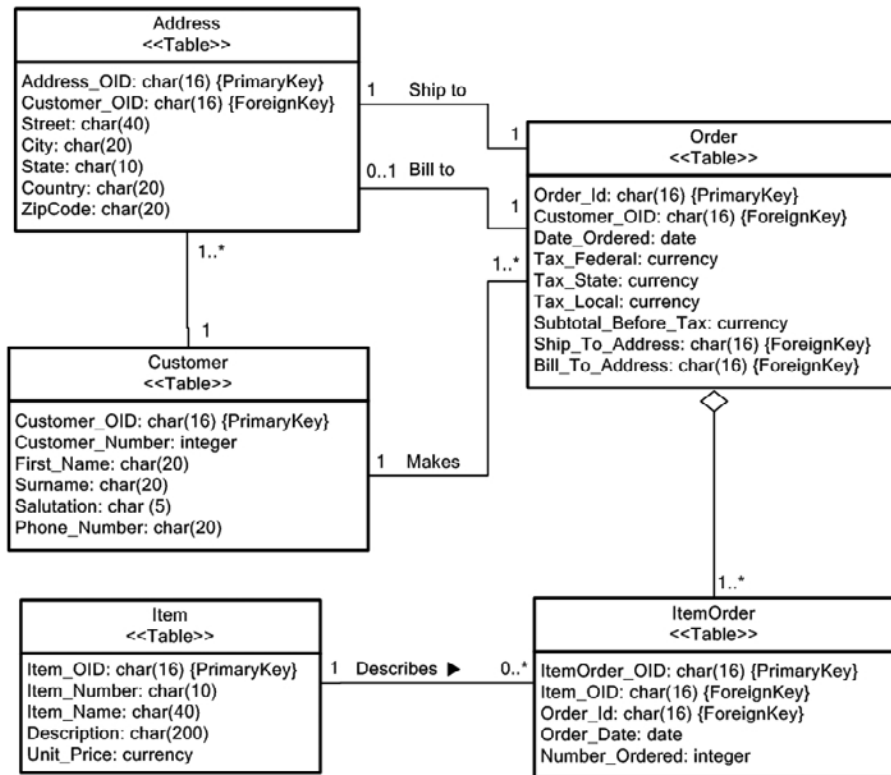


Figure 5.2 A start at a physical data model for SWA Online.

something that takes time and effort. A good way to start is to focus on a core subset of them at first, and then learn new techniques one at a time as needed. Methodologies such as ICONIX (Rosenberg and Scott 1999) and Catalysis (D'Souza and Wills 1999) will often recommend several artifacts and describe how to apply them in practice, and then as you discover missing aspects of the methodology you can supplement it with the appropriate modeling artifacts.

Create Several Models in Parallel

Because each type of model has its strengths and weaknesses, no single model is sufficient for your modeling needs. For example: When you are exploring requirements, you may need to decide to develop a use case (Jacobson et. al. 1992; Cockburn 2001a) as you see in Figure 5.3; an essential UI prototype as you see in Figure 5.4 (Constantine and Lockwood 1999); and Class Responsibility Collaborator (CRC) cards (Cunningham and Beck 1989) as you see in Figure 5.5. The use case describes how someone places an order, the UI prototype specifies the requirements for a screen or page to support order entry, and the CRC cards capture conceptual information about the business domain.

- The use case begins when a customer chooses to place an order.
- The customer searches for items via the use case "Search for Item(s)."
- The customer adds an order item to their order.
- The customer indicates the number of a given item they wish to order.
- The system calculates the subtotal for the item by multiplying the unit price by the number ordered.
- The customer repeats steps 2 through 5 as necessary to build their order.
- The customer finishes adding items to their order.
- The customer provides their ship to and bill to information, including their name, phone number, and surface address.
- The system calculates the subtotal for the entire order by adding the subtotals of the individual line items.
- The system calculates the taxes applicable for the order according to the business rule *Calculate Taxes for an Order*.
- The system calculates applicable discounts for the order according to the business rule *Calculate Discounts for an Order*.
- The system displays the applicable taxes and discounts.
- The system calculates the grand total for the order by adding the applicable taxes to the order subtotal and subtracting the discounts.
- The system displays a summary of the order.
- The customer verifies that the order is what they want.
- The system schedules the order for fulfillment (see the use case *Fulfill Order*).
- The system produces a receipt for the customer summarizing the order.

Figure 5.3 The basic course of action for placing an order.

As you work with your project stakeholders to explore their requirements, you would update each of these models appropriately, capturing information in the artifact in which it makes the most sense. Similarly, when you are designing Java software, you may discover that you need to develop a UML class diagram to formulate its structure, a UML state chart diagram to explore the inner workings of a complex class, a UML sequence diagram to determine how to implement the logic of a flow within a use case (as you see in Figure 5.1) or a business rule, and a physical data model to understand the structure of your relational database as you see in Figure 5.2. In combination with the practice of *Iterate to Another Artifact* (see next section), agile modelers will often discover that they are far more productive working on several models simultaneously than if they are focusing on only one at any given time.

An interesting implication of this practice is that it brings into question two common antipatterns in the IT industry. The first one is something that I call Single Artifact Developers, someone who specializes in the development of one kind of

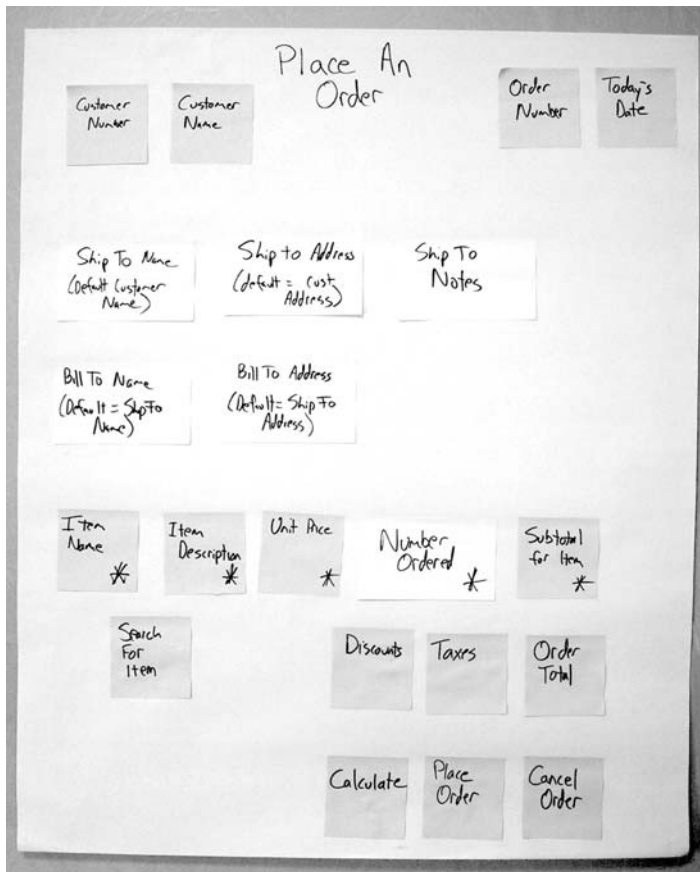


Figure 5.4 An essential UI prototype depicting the requirements for a screen/page.

deliverable. Examples of this are people who want only to write code, low-end coders who believe that everything of importance is in the source code and nothing else matters, and data modelers who believe that the most important thing is data and everything else pales in comparison. The reality is that modern software is far too complex for anyone with such a narrow focus to be effective. My advice is to become very adept at creating several types of artifacts, source code and data models are actually good candidates, but to have at least a passing understanding of a wide range of techniques. In other words, fill your intellectual toolbox as best you can. The second antipattern is Single Artifact Modeling Sessions. Use case modeling sessions are the most common example, where you work on one type of model. A requirements modeling session makes sense to me, a use case modeling session just doesn't make any sense in an agile environment.

Creating several models in parallel will be difficult at first for some people, particularly those who prefer to focus on a single task at a time and those who currently exhibit the Single Artifact Developer antipattern. If AM is new to your organization,

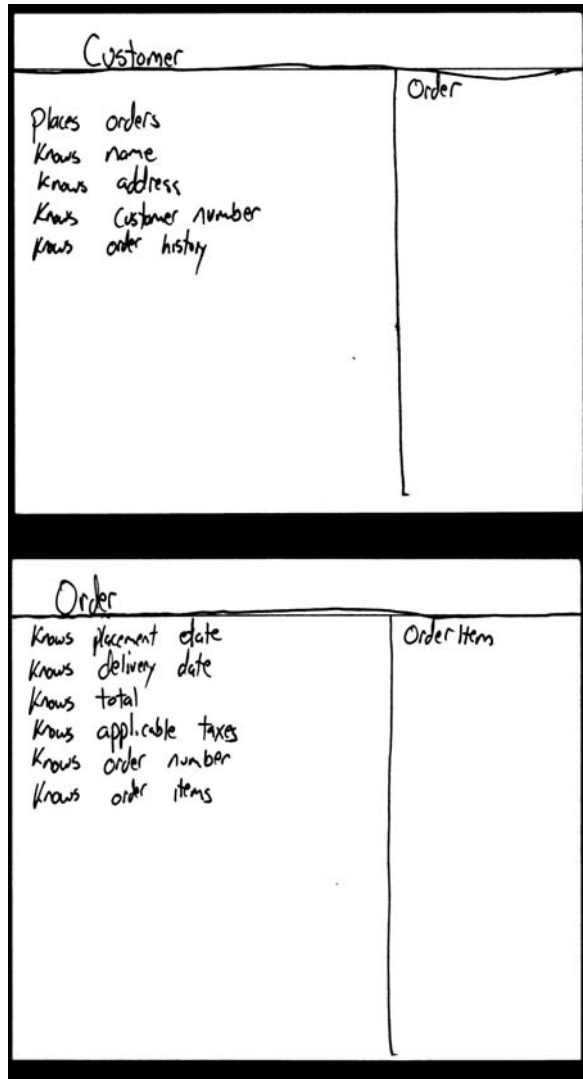


Figure 5.5 Two CRC cards for SWA Online.

then you need to be prepared to help people get used to practices such as this one. I have found that the first step to overcome these difficulties is to work on two models at once, following the practice *Model with Others* discussed later in this chapter and having someone involved who is comfortable with this approach work with the person who is uncomfortable. I will usually focus on an artifact they are familiar with and one that they would usually consider “downstream” that they are also familiar with. Perhaps I would choose a use case and a UML sequence diagram in combination, or a UML class diagram and a physical data model. The goal is to help them build up their confidence in the practice, to prove to them that it does in fact work well.

Iterate to Another Artifact

Perhaps you are working a use case, CRC card, sequence diagram, or even source code and discover you are struggling with it. This is an indication that you should consider working on another artifact for the time being. Each artifact has its strengths and weaknesses; each artifact is good for a certain type of job. Whenever you find you are having difficulties working on one artifact—perhaps you are working on a use case and find that you are struggling to describe the business logic—then that’s a sign that you should iterate to another artifact. By iterating to another artifact, you immediately become “unstuck” because you are making progress working on that other artifact. Furthermore, by changing your point of view you often discover that you address whatever it was that caused you to be stuck in the first place. Besides regaining momentum, you get the added bonus of getting meaningful work accomplished, a key concept for agile software development.

The hardest thing when applying this practice, after recognizing that you are in fact stuck, is to identify a likely candidate to iterate to. Experience will guide your choice, but if you are short on experience, then in Appendix A I provide an indication of likely candidates to iterate to when you are working on a modeling artifact. For example, if you work on an essential use case, then you may want to consider changing focus to start working on an essential UI prototype, a CRC model, a business rule, a system use case, or a change case.

Model in Small Increments

The principle *Incremental Change* indicates that an incremental approach to development is a fundamental of AM. In fact, you saw in Chapter 1, “Introduction,” that it is a fundamental aspect of agile software development in general (Agile Alliance 2001b). The basic idea is that you organize a larger effort into smaller portions that you release over time, ideally in increments of several weeks or a month or two. This increases your agility by enabling you to deliver software into the hands of your users faster, and thus obtain concrete and rapid feedback from them throughout your project. Because you are taking an incremental approach to development, you will also be taking an incremental approach to modeling.

With incremental development you model a little, code a little, test a little, then deliver a little. No more big design up front (BDUF), where you invest weeks or even months creating models and documents. Instead, the majority of modeling sessions, impromptu gatherings of people to explore one or more issues, last on the order of 10 or 20 minutes. Agile modelers model in small increments, just long enough so they can return to working on software as outlined in the principle *Software is Your Primary Goal*. Longer modeling sessions, sometimes lasting several days (particularly at the beginning of a project), can occur but they are the exception, not the norm. The longer you go without concrete feedback, the greater the chance that what you are modeling won’t be what is actually needed and therefore will be wasted effort. Furthermore, you run the risk of violating the practices of *Depict Models Simply* and *Create Simple Content*, described later in this chapter, when you focus for a long time on your model(s).

The hardest part of following this practice is to stop modeling once you've fulfilled your goal. You often want to model the things that you'll need to work on tomorrow, next week, or next month. Your subconscious goal is to think things through as best you can. Stop doing this. Have the courage to solve today's problem today, and trust that you can solve tomorrow's problem tomorrow (Beck 2000).

Practices for Effective Teamwork

AM defines four practices that enable effective teamwork and communication within your team and with your project stakeholders:

1. Model with Others
2. Active Stakeholder Participation
3. Collective Ownership
4. Display Models Publicly

Model with Others

Software development is a lot like swimming, it's very dangerous to do it alone. When you model with a purpose, you often find that you are modeling to understand something or that you are modeling to communicate your ideas to others to develop a common vision regarding your project. These are best suited as group activities because you want the input of several people working together effectively. You will often find that your development team needs to work together to create the core set of models that are critical to your project. For example: To develop the metaphor or architecture for your system, you will often need to model with a group of people to develop a solution everyone agrees on, as well as one that is as simple as possible. Most of the time the best way to do this is to talk the issue through with one or more people. There's nothing wrong with drawing a simple sketch to think something through on your own, but once you are finished, talk about your ideas with someone to see if you're going in the right direction. Two or more heads are better than one. This practice helps to improve communication on your project, helps to build a common vocabulary among the people you are working with, increases the chance that you'll do quality work, and provides opportunities for people to learn from their co-workers.

This practice can be difficult to adopt at first within organizations that have a "divide and conquer" culture where the focus is on assigning work to individuals, or that have highly competitive cultures that pit individuals against each other. To counteract the problem you will need to communicate the benefit of working together, perhaps taking the approach that "new methodologies imply new ways of working so let's just try it for now and see what happens." You also need a work area, see Chapter 11, "Agile Work Areas," that supports people working together as a team. This may be something as simple as a whiteboard on the wall of someone's cubicle or a dedicated workspace for your team.

Active Stakeholder Participation

A project stakeholder is anyone who is a direct user, indirect user, manager of users, senior manager, operations staff member, support (help desk) staff member, tester, developer working on other systems that integrate or interact with the one under development, or maintenance professional potentially affected by the development and/or deployment of a software project. For the sake of AM, in this definition I exclude developers who are working on the project—even though developers clearly have an important stake in the projects that they work on for AM, the term “project stakeholder” will be used to refer to everyone with a stake in a project except for the developers working on it. Hence I refer to developers and project stakeholders as separate groups of people. The alternative is to have terms such as developer project stakeholders and non-developer project stakeholders (yuck).

AM’s practice of *Active Stakeholder Participation* is closely related to its practice *Model with Others*. Furthermore, it is an expansion of eXtreme Programming (XP)’s *On-Site Customer* (Beck 2000) practice that describes the need to have on-site access to people, typically users or their representatives, who have the authority and ability to provide information pertaining to the system being built and to make pertinent and timely decisions regarding the requirements and prioritization thereof. While this level of participation is required to make your software development efforts effective, it often isn’t sufficient in many organizations, particularly those where politics and not a true commitment to building working systems are the order of the day. Project success often requires a greater level of involvement by project stakeholders: Senior management needs to publicly and privately support your project, operations and support staff must actively work with your project team towards making your production environment ready to accept your system, other system teams must work with yours to support integration efforts, and maintenance developers must work to become adept at the technologies and techniques used by your system. This practice is motivated by the principles of *Rapid Feedback* and *Open and Honest Communication*.

It is clear that in order to be successful, all project stakeholders must actively work with your team to achieve these goals. There are several implications of this practice:

- Users must be prepared to share business knowledge with the team and to make both pertinent and timely decisions regarding project scope and requirement priorities.
- For senior managers to effectively support your project, they must first understand the benefits and added value of the technologies and techniques that your team is using, understand why your team is using them, and understand the implications of using them. With this knowledge, their efforts within your organization’s political arena are far more likely to be effective at the right times in the right ways. Senior managers won’t be able to gain this requisite knowledge by reading a weekly project status report or by attending a monthly project steering meeting. Instead, they need to invest the necessary time to learn about the things that they manage; they need to actively participate in the development of your system.

- Your operations and support organization must invest the resources required to understand both your system and the technologies that it uses. Your support staff must take the time to learn the nuances of your system; the implication is that they need to work with your system as it is developed and/or your team will need to provide them with training. Furthermore, your operations staff must become proficient with both the installation and operation of your system. You may choose to include one or two operations engineers on your development team or once again to invest project resources to train operations staff as required. Regardless of your approach, both your operations and support organizations will need to be actively involved with your project team.
- Other project teams need to work with you if your system needs to integrate with other systems. For example: Perhaps your system needs to access a legacy database, interact with an online system, work with a data file produced by an external system, or provide an XML data extract for other systems. Integration often proves difficult if not impossible without the active participation of these developers. Imagine how difficult it would be to access the information contained in a large legacy database if the owners of that database refuse to provide any information about it. Remember that communication is a two-way street; you'll also be sharing information about your system with other teams.
- Maintenance developers need to work with you to learn your system. When the intention is to either partially or completely hand off the maintenance of your system to other developers, it is common to bring in software professionals skilled in maintaining and enhancing existing systems to free up members of the original development team; then your team must work with these people so they can take over the system from you. Even when some original team members are still involved, you must make an effort to transfer the knowledge to the new members of the team. A good example of this is having original team members mentor new members or simply pair up with them when working on new aspects of the system.

Collective Ownership

Everyone can work on any model, and ideally any artifact on the project, if they need to. For example: If I draw a data flow diagram (DFD) on a whiteboard, there are several advantages to this approach. First, the more people who get involved with the development of an artifact, the greater the opportunity to identify potential issues with it, supporting the principle of *Rapid Feedback*. Second, it provides people with opportunities to gain experience developing various types of models, expanding their intellectual toolbox, and thus making them more effective as agile modelers—supporting the principle that *Everyone Can Learn From Everyone Else* because they can see each other's work and even improve upon it. Third, it reduces the temptation for team members to say things like “Your model is wrong,” because if they discover that something is wrong, they should fix it, not complain about it. Fourth, it reduces the chance that people will personalize certain artifacts, such as: “This class model is my baby and

nothing could possibly be wrong with it,” because no single artifact is only theirs. Fifth, it promotes understanding of the system among team members, improving communication within your team and reducing both your need to maintain extensive documentation and your reliance on a single person or subset of people. There is a project management concept called truck number. The idea is that it is an estimate of the minimum number of people you would need to lose from your team before you find yourself in trouble (for example, the number of people that would need to be hit by a truck). A truck number of one is a serious problem; a truck number greater than or equal to the number of people on your team is ideal. Collective ownership increases your project’s truck number.

This practice can be challenging to adopt within organizations where the individual is more important than the team, and/or that have a focus on narrowly defined roles on a project team. People need to accept that the artifacts that they work on are the property of the team. They also need to work on a wide variety of things—not work just on the user interface, or just on a single subsystem, or just on system integration code.

Display Models Publicly

You should display your models publicly, often on something called a “modeling wall” or a “wall of wonder” (Gottesdiener 2001). This supports the principle of *Open and Honest Communication* on your team because all of the current models are quickly accessible to them, as well as with your project stakeholders because you aren’t hiding anything from them. Your modeling wall is where you post your models for everyone to see; the modeling wall should be accessible to your development team and other project stakeholders. Your modeling wall may be physical, perhaps a designated whiteboard for your architecture diagram(s) or a place where you tape a printout of your physical data model. Modeling walls can be virtual, such as an internal Web page that is updated with scanned images.

A further benefit of this practice is that it shows to your project stakeholders that you are doing valuable work—it’s right there in front of their eyes. This is particularly good when you are developing simple models for the first time and are afraid that you might not be adding the same value (due to lack of quantity) that you did in the past. Having several simple models displayed on the wall that are publicly being used goes a long way to showing the worth of your contributions.

This practice can be difficult to implement in firms where wall space is at a premium, where the project team is working in a relatively public area that has your firm’s customers or even competitors walking through it and you don’t wish them to see your work in progress, or in an environment where significant investment has been made in wall decorations (such as a law firm with oak-covered walls) that you don’t wish to harm. If any of these problems are the case, you may wish to consider moving your team to another location. Furthermore, cultural issues such as the unwillingness to share information with people outside your group can also hamper this practice. If this is the case, then my advice is to find the courage to adopt this practice.

Practices That Enable Simplicity

AM defines three practices that enable simplicity within your modeling efforts:

1. Create simple content
2. Depict models simply
3. Use the simplest tools

Create Simple Content

You should keep the actual content of your models—your requirements, your analysis, your architecture, or your design—as simple as you possibly can while still fulfilling the needs of your project stakeholders. The implication is that you should not add additional aspects to your models unless they are justifiable. For example: The UML class diagram (Rumbaugh, Jacobson, and Booch 1999) in Figure 5.6 does not indicate the visibility of the attributes and operations of the classes, presumably something that whoever will be programming these classes (hopefully the people who drew the diagram) will determine when they get to it. This is along the lines of XP's practice of

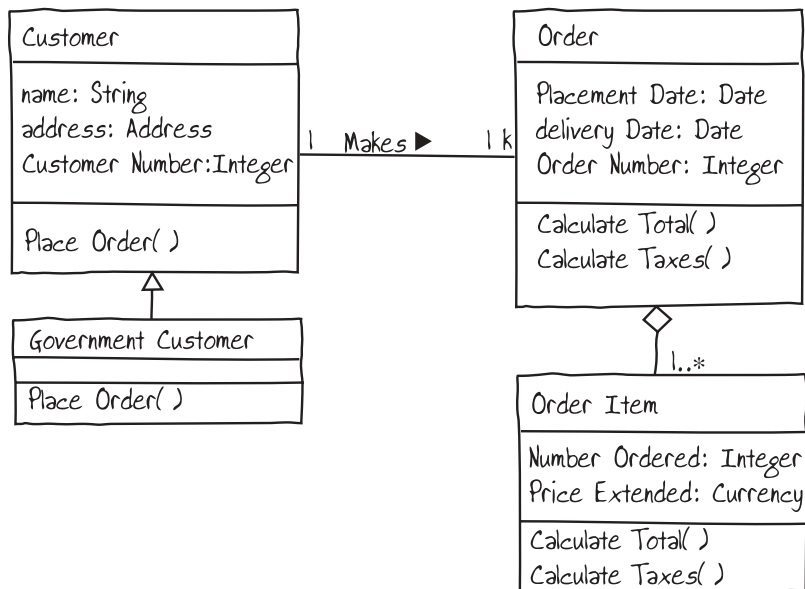


Figure 5.6 A UML class diagram.

Simple Design (Beck 2000). This practice is also applicable to non-model artifacts such as source code, project plans, and user documentation.

So how do you know when the content of your models is simple? I believe the following factors, modified from Kent Beck's (2000) simplest design advice, for determining when a model is simple:

- The model communicates everything that you want to communicate. In other words, it fulfills its purpose.
- The model must contain no duplicate information.
- The model should have the fewest possible elements.

The most common stumbling blocks to adopting this practice are your tendency to want to over-model, something that you can overcome by recognizing the problem and stopping yourself when your model has fulfilled its immediate purpose, and the tendency of others within your organization to equate progress to detailed models and documentation. You'll often find significant organizational peer pressure to conform to existing standards of documentation and modeling detail, particularly in firms with prescriptive processes that are new to agile approaches. If this is the case, you will need to communicate your approach to others within your organization, explaining the reasons behind what you are doing. You should even consider lending them your copy of this book or, better yet, suggest that they buy their own copy!

Depict Models Simply

When you consider the potential diagramming notations that you could apply (UML diagrams, user interface diagrams, data models, and so on), you quickly realize that most of the time you require only a subset of the diagramming notation available to you. For example: A simple model that shows the key features that you are trying to understand, perhaps a class model depicting the primary responsibilities of classes and the relationships between them, often proves to be sufficient. Yes, you could model all the scaffolding code that you will need to write, all the getter and setter operations that your coding standards tell you to use, but what value would that add? Very little for an agile modeler.

Although this practice complements *Create Simple Content*, the two concepts are orthogonal. *Create Simple Content* focuses on the subject matter of the model, whereas *Depict Models Simply* focuses on how you present your models. Common techniques (Ambler 2002) to simplify your diagrams include:

- Avoid crossing lines
- Avoid curved lines
- Avoid diagonal lines
- Avoid different size bubbles
- Avoid too many bubbles (no more than 7 +/- 2)
- Avoid unnecessary detail

Use the Simplest Tools

The vast majority of models can be drawn on a whiteboard, on paper, or even on the back of a napkin. Whenever you want to save one of these diagrams, you can take a picture of it with a digital camera or even transcribe it onto paper. In fact, you've seen several models created with simple tools. Figure 5.1 was created using flipchart paper, sticky notes, and markers; Figure 5.2 was drawn on a whiteboard; and the CRC cards of Figure 5.3 were created with index cards and a pen. Using the simplest tools works because most diagrams are throwaways. Their true value comes from drawing them to think through an issue, and once the issue is resolved, the diagram doesn't offer much value. As a result, a whiteboard and markers are often your best modeling tool alternative. Use a drawing tool to create diagrams to present to important project stakeholders and occasionally use a modeling tool if and only if they provide value to programming efforts such as the generation of code. Think of it like this: If you're creating simple models, often models are throwaways because if you are modeling to understand, you likely don't need to keep them once you do understand the issue; therefore, you don't need to apply a complex modeling tool.

Chapter 10, "Using the Simplest Tools Possible?" explores this practice in detail. Although I explicitly pointed this out in Chapter 1, I'm going to do so again: AM has nothing against CASE tools. If investing in a CASE tool is the most effective use of your resources, then by all means do so, and then use it to the best of its ability. My experience is that there are many CASE tools on the market but that few of them are worth the bother. If a simple tool is sufficient for your needs, then use it.

This can be a hard practice to adopt within organizations that are accustomed to models created using sophisticated tools. Many developers equate modeling with using an expensive CASE tool, and it's very difficult for them to accept that a stack of index cards can be effective. The best way to address this situation is to get them actively involved with using the simple tools, giving them experience in techniques such as CRC modeling and essential UI prototyping.

Practices for Validating Your Work

AM defines two practices that pertain to the testing and validation of your work:

1. Consider testability
2. Prove it with code

Consider Testability

When you model, you should constantly ask yourself, "How are we going to test this?" If you can't test the software that you build, you shouldn't build it. Modern software processes include testing and quality assurance activities throughout the entire project lifecycle, and some even promote the concept of writing tests before writing software, an XP practice called Test-First Design (Beck 2000). Agile developers test early and test often, ensuring that they are producing quality work. The hardest aspect

of adopting this practice is to learn the habit of constantly thinking “in the background” about how you’re going to test your work.

Prove It with Code

A model is an abstraction, one that should accurately reflect an aspect of whatever you build. To determine if it will actually work, you should validate it by writing the corresponding code. You’ve developed a sketch of an HTML page for accepting billing address information? Code it and show the resulting user interface to your users for feedback. You’ve developed a UML sequence diagram that represents the logic to implement a complex business rule? Write the testing code, the business code, and then run the tests to ensure that you’ve gotten it right. Never forget that with an iterative and incremental approach to software development, the norm for the vast majority of projects, modeling is only one of many tasks that you will perform. Do some modeling, do some coding, and do some testing (among other things). Although the focus of AM is on modeling, never forget that there is far more to development.

There are several common impediments to this practice.

- It works best when the people doing the modeling are also the ones writing the code, implying that an agile developer needs a wide range of skills. In Chapter 12, “Agile Modeling Teams,” I argue that agile developers need to be generalists with one or more specialties. Remember, there is far more to software development than modeling.
- Second, many developers have a “big modeling up front” (BMUF) mindset that leads them to model for greater periods of time than they need to, putting off coding for a while.
- Third, many developers are accustomed to a process where you create a model, review and rework it, then code it. With AM you are much more likely to do a little modeling and then do a little coding. The need to review your model goes away because you’re proving your model with code and because you are usually your model(s)’ customer and therefore are the one directly affected by any problems with it (motivating you to do quality work to begin with).